PostgreSQL RDS/Aurora DBA Learning Path

A comprehensive, production-grade curriculum for PostgreSQL DBAs and cloud database engineers managing RDS and Aurora in AWS environments. This guide integrates current 2024–2025 best practices across instance management, performance tuning, high availability, security, and troubleshooting — everything from day-to-day operations to complex architectural decisions.

13 CORE MODULES | PRODUCTION READY | 2024–2025 BEST PRACTICES

Learning Path Overview

This curriculum is structured as a progressive learning journey — from foundational instance management through expert-level troubleshooting and production architecture design. Each module builds on the last, ensuring a solid operational foundation before advancing to complex distributed systems concepts.

Foundation

Instance creation, parameter groups, basic monitoring — the operational bedrock every DBA must master.

Operations

Backup/restore, patching, minor upgrades, and routine maintenance workflows.

Performance

Query tuning, indexing strategies, and autovacuum management for production workloads.

HA/DR

Multi-AZ configurations, read replicas, failover testing, and disaster recovery planning.

Advanced → Expert

Aurora architecture, Serverless v2, Global Database, complex migrations, and production architecture design.

MODULE 1

Instance & Cluster Management

Understanding the architectural differences between RDS PostgreSQL and Aurora PostgreSQL is the most fundamental skill for cloud DBAs. These two services share the PostgreSQL engine but diverge significantly in storage architecture, failover behavior, and scalability characteristics. Choosing the wrong service for a workload can result in cost inefficiency or unacceptable downtime.

Aspect	RDS PostgreSQL	Aurora PostgreSQL
Architecture	Single instance + Multi-AZ standby	Shared storage, writer + up to 15 readers
Max Storage	64 TiB (gp3/io1/io2)	128 TiB (distributed)
Storage Types	gp2, gp3, io1, io2	Aurora-native (I/O-Optimized available)
Failover Time	1–2 minutes	~30 seconds (10–15s DNS + 3–10s recovery)
Serverless Option	Not available	Aurora Serverless v2 (0.5–128 ACUs)

2024–2025 Key Updates: gp3 is now the default storage type for new RDS instances, delivering 3,000 IOPS baseline independent of volume size. Multi-AZ with two readable standbys now supports gp3. Aurora I/O-Optimized can provide up to **40% cost savings** when I/O exceeds 25% of total database spend.

MODULE 2

PostgreSQL Configuration: RDS & Aurora

Parameter tuning on managed PostgreSQL differs significantly from self-managed instances. RDS exposes most GUC parameters through parameter groups, while Aurora manages certain memory parameters — notably `shared_buffers` — dynamically at the engine level. Understanding which parameters you control versus which the service manages is critical to avoiding wasted tuning effort.

Parameter	RDS Default	Aurora Default	Recommendation
`shared_buffers`	~25% of RAM	Managed by Aurora	Monitor BufferCacheHitRatio >99%
`work_mem`	4MB	4MB	Increase for complex sorts/joins
`maintenance_work_mem`	64MB	64MB	Increase for VACUUM/CREATE INDEX
`effective_cache_size`	75% of RAM	75% of RAM	Query planner hint

Enable pg_stat_statements

Add to `shared_preload_libraries` via parameter group. Requires an instance restart. Essential for identifying top resource consumers.

Session Timeout Controls

Set `idle_in_transaction_session_timeout` to 5–15 minutes. Apply `statement_timeout` at the role level, not just in application code.

Autovacuum Visibility

Log autovacuum minimum duration at 10 seconds or lower. Log temp file usage when approaching limits to catch runaway sort operations.

MODULE 3

High Availability & Failover

High availability architecture is one of the most consequential decisions in cloud database design. Both RDS and Aurora offer robust HA options, but their failure modes, recovery times, and operational implications differ enough to warrant careful evaluation against your RTO requirements.

RDS Multi-AZ Options

One Standby (Traditional) — synchronous replication, no read traffic on standby. Failover in 1–2 minutes.

Two Readable Standbys (Multi-AZ Cluster) — semi-synchronous replication, can serve reads, up to 2x faster writes compared to single-standby.

Aurora HA Architecture

- 6-way replication across 3 AZs (4/6 write quorum, 3/6 read quorum)
- Cluster Cache Management — warm cache preserved on failover
- Backtrack — rewind up to 72 hours without a full restore
- ~30 second failover (10–15s DNS propagation + 3–10s recovery)

Failover Optimization with RDS Proxy

RDS Proxy is a fully managed connection proxy that significantly improves failover resilience. It preserves idle connections during failover events, handles active transaction interruption gracefully for clients, and bypasses DNS TTL delays — delivering up to **66% faster recovery** compared to direct connections. RDS Proxy is essentially required for Aurora Serverless v2 connection management due to the scale-up latency between ACU transitions.

MODULE 4

Replication & Read Scaling

Read scaling strategies differ fundamentally between RDS and Aurora, and choosing the right architecture has significant implications for replication lag, storage costs, and impact on the primary instance. Aurora's shared storage model eliminates WAL shipping overhead entirely, making it the superior choice for read-heavy workloads requiring many reader endpoints.

Feature	RDS Read Replicas	Aurora Reader Nodes
Max Replicas	5 per instance, 155 total (3 levels)	15 per cluster
Replication Lag	Seconds to minutes	Few hundred milliseconds
Cross-Region	5 replicas	Aurora Global Database
Storage	Independent EBS volumes	Shared storage (no replay lag)
Impact on Primary	Significant (WAL shipping)	Negligible

Monitoring Replication Health

For RDS read replicas, use `pg_stat_replication` on the primary to track write/flush/replay lag per replica. On the standby, query `pg_stat_database_conflicts` to identify hot standby conflicts that may indicate long-running queries blocking replay. Set a CloudWatch alarm on `ReplicaLag` with a threshold of 30 seconds.

```
-- Monitor replication lag
SELECT client_addr, state, sent_lsn, write_lsn,
 flush_lsn, replay_lsn,
 write_lag, flush_lag, replay_lag
FROM pg_stat_replication;

-- Check for conflicts on standby
SELECT * FROM pg_stat_database_conflicts;
```

MODULE 5 ⭐ CRITICAL DBA SKILL

Backup & Recovery

Backup and recovery is the single most critical operational skill for any DBA. A backup strategy that has never been tested is not a backup strategy — it is a hope. Production DBAs must understand the backup mechanisms of both RDS and Aurora, their performance implications, and the recovery speed differences before an incident occurs, not during one.

Aspect	RDS PostgreSQL	Aurora PostgreSQL
Backup Type	Daily full + continuous WAL	Continuous incremental
Performance Impact	Slight on single-AZ	None
PITR Speed	Slower (restore + WAL replay)	Fast (incremental, storage-level)
Retention Range	0–35 days	1–35 days

Minimum Retention

Set at least 30 days for both automated snapshots and logical backups. This satisfies most compliance requirements and provides adequate recovery windows.

Regular Restore Testing

Validate backups quarterly by restoring to a separate environment. Test both PITR and snapshot restores. Document the time-to-restore for your RTO/RPO planning.

Cross-Region DR Copies

Copy snapshots to a secondary region for disaster recovery. Logical backups via `pg_dump` provide a secondary layer of protection against snapshot corruption.

Post-Upgrade ANALYZE

After any major version upgrade, run `ANALYZE` on all tables immediately. Statistics are not automatically carried forward and stale stats cause poor query plans.

Aurora Continuous Backup: Aurora uses storage-level incremental snapshots with no performance impact during backup windows. Crash recovery is instant — no checkpoint replay is needed, unlike RDS which must replay WAL from the last checkpoint on recovery.

MODULE 6

Performance Tuning

Performance tuning on managed PostgreSQL requires a methodical, evidence-based approach. Start with `pg_stat_statements` to identify the top resource consumers before touching any configuration. The most impactful tuning interventions are almost always query and index improvements — not memory parameter changes.

Top SQL by Resource Consumption

```
-- Enable pg_stat_statements (requires restart)
shared_preload_libraries = 'pg_stat_statements'

-- Top resource-consuming queries
SELECT query, calls, total_exec_time, mean_exec_time, rows,
    100.0 * shared_blks_hit /
    nullif(shared_blks_hit + shared_blks_read, 0) AS hit_percent
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

Index Anti-Patterns to Eliminate

Redundant Secondary Indexes

Indexes that duplicate a prefix of another index consume write overhead with no query benefit. Audit with `pg_stat_user_indexes` and drop those with zero scans.

Individual vs. Composite Indexes

Multiple single-column indexes on a frequently joined table are usually worse than a well-designed composite index covering the common access pattern.

Low Cardinality & Full-Length String Indexes

Indexes on boolean or low-cardinality columns are rarely useful. Use partial indexes on subsets of data. For long strings, index only a functional prefix or hash.

Scaling Guidelines

100s	100...	GBs	<10
Connections	**Tables**	**Relation Size**	**Indexes / Table**
Measure in hundreds, not thousands. Beyond ~500 active connections, performance degrades rapidly without pooling.	Schema table count should measure in thousands, not hundreds of thousands. Excessive tables stress `pg_class` and autovacuum scheduling.	Individual relation sizes should be measured in GBs. TB-sized tables signal the need for partitioning.	Keep indexes per table in single digits. Each additional index adds write amplification on INSERT/UPDATE/DELETE.

MODULE 7

Monitoring & Observability

Effective observability for managed PostgreSQL requires a layered approach: AWS-native tools (Performance Insights, CloudWatch, Enhanced Monitoring) provide the infrastructure and database-level signal, while PostgreSQL's own catalog views (`pg_stat_statements`, `pg_stat_activity`, `pg_stat_replication`) provide the query-level detail needed to diagnose root causes.

Performance Insights: Key Concepts

DBLoad Metric

Measures average active sessions (AAS). Values above vCPU count indicate the database is CPU-bound and queries are queuing. DBLoad is the most important single metric for diagnosing performance degradation.

Top SQL by Wait Events

Performance Insights decomposes DBLoad by SQL statement and wait event type (CPU, Lock, IO, etc.). This allows you to pinpoint exactly which queries are contributing to load spikes.

AAS / vCPU Ratio

Compute AAS = DBLoad / vCPU count. A ratio consistently above 1.0 means queries are waiting. Ratios above 2.0 indicate a serious performance problem requiring immediate investigation.

Essential CloudWatch Alert Thresholds

Metric	Alert Threshold	Purpose
`DatabaseConnections`	>80% of max	Connection saturation — implement pooling
`CPUUtilization`	>80% sustained	Compute bottleneck — scale up or optimize
`FreeableMemory`	<10% of total	Memory pressure — risk of OOM events
`ReadLatency / WriteLatency`	>20ms	Storage performance degradation
`ReplicaLag`	>30 seconds	Replication health alert

Enhanced Monitoring: Enable OS-level metrics at 1–10 second granularity for deep-dive investigations. Retain Enhanced Monitoring data for a minimum of 1 month to support trend analysis and incident post-mortems.

MODULE 8

Maintenance Operations

Autovacuum is PostgreSQL's housekeeping process responsible for reclaiming dead tuple storage, updating visibility maps, and preventing transaction ID wraparound. On managed RDS/Aurora, autovacuum runs continuously in the background — but its default configuration is tuned for small-to-medium tables. Large, high-churn production tables almost always require table-level overrides to prevent bloat accumulation and wraparound emergencies.

Autovacuum Tuning per Table

```
-- Table-level autovacuum settings for high-churn tables
ALTER TABLE large_table SET (
  autovacuum_vacuum_scale_factor = 0.1,   -- Default 0.2 (20%)
  autovacuum_vacuum_threshold    = 1000,  -- Default 50
  autovacuum_analyze_scale_factor = 0.05,
  autovacuum_max_workers         = 3
);

-- For insert-only tables (time-series/events)
ALTER TABLE events SET (
  autovacuum_freeze_min_age = 10000000,
  fillfactor = 100
);
```

Bloat Detection

```
-- Check table bloat
SELECT schemaname, tablename, n_dead_tup, n_live_tup,
      round(n_dead_tup::numeric/nullif(n_live_tup,0)*100, 2) AS dead_pct
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;
```

Avoid VACUUM FULL

VACUUM FULL acquires an exclusive lock for the duration of the rebuild. Use `pg_repack` for online table reconstruction without blocking production traffic.

CREATE INDEX CONCURRENTLY

Always use `CONCURRENTLY` when rebuilding bloated indexes in production. It takes longer but avoids the full table lock that blocks all DML.

Manual VACUUM FREEZE

For insert-only tables (logs, events), autovacuum may not trigger freeze often enough. Schedule manual `VACUUM FREEZE` runs to prevent XID wraparound.

MODULE 9

Connection Management

Connection management is one of the most overlooked performance dimensions in PostgreSQL deployments. PostgreSQL uses a process-per-connection model — each connection spawns a backend process with its own memory allocations. At high connection counts (thousands), context switching overhead, shared memory contention, and scheduler pressure combine to degrade throughput significantly. Production systems should target hundreds of active connections, not thousands.

Instance Connection Limits

Connection limits scale with instance class. A `db.t3.micro` supports approximately 80 connections, while a `db.r6g.16xlarge` supports up to ~5,000. Aurora Serverless v2 provisions approximately 1,000 connections per ACU — so a cluster at 4 ACUs can handle ~4,000 connections. Plan connection budgets as part of capacity planning, not as an afterthought.

Connection Pooling Strategy

Layer	Tool	Use Case	Notes
Application	HikariCP, PgBouncer	Standard long-running apps	Low latency, in-process pooling
Infrastructure	RDS Proxy	Serverless, Lambda, failover	Managed, IAM-integrated, multiplexing
Database	PgBouncer on EC2	Complex pooling needs	Transaction-mode for aggressive pooling

RDS Proxy Multiplexing

One proxy connection can serve many database connections. Reduces the number of actual backend processes, lowering per-connection memory overhead and improving throughput under burst load.

IAM Authentication Integration

RDS Proxy supports IAM-based authentication, enabling short-lived token-based access without storing long-lived credentials in application configuration or environment variables.

Serverless v2 Connection Storms

Set minimum ACU based on connection requirements, not just cost targets. Each scale-up step has latency. Use RDS Proxy to buffer connection spikes during the brief scale-up lag window.

MODULE 10

Security: DBA-Level Hardening

Security hardening for managed PostgreSQL in AWS encompasses network isolation, authentication, encryption, access control, and audit logging. Each layer independently reduces attack surface — a defense-in-depth posture ensures that a compromise at one layer does not result in full data exposure. DBAs are responsible for the database-layer controls; network and IAM controls should be implemented in coordination with your security team.

Feature	Implementation
IAM Authentication	Token-based, auto-rotates every 15 minutes — no long-lived passwords for applications
SSL Enforcement	Set `rds.force_ssl=1` in the parameter group; verify via `pg_stat_ssl`
Encryption at Rest	KMS-managed (default AES-256); enable at creation — cannot be added post-creation
Column Encryption	`pgcrypto` extension for application-layer column-level encryption
Row-Level Security	Native PostgreSQL RLS policies — enforce at the table level, not application logic
Audit Logging	`pgAudit` extension + Database Activity Streams (Aurora) for real-time audit

Deploy in Private Subnets Only

RDS/Aurora instances should never be placed in public subnets. Use VPC endpoints for AWS service access and bastion hosts or AWS Systems Manager Session Manager for administrative access.

Use Secrets Manager for Credential Rotation

Configure automatic rotation for database master credentials and application users. Secrets Manager integrates natively with RDS and Aurora, enabling zero-downtime credential rotation.

Grant Least Privilege — No Master Credentials for Apps

Create dedicated application roles with only the permissions required. Never use the RDS master user in application connection strings. Master credentials should be reserved for DBA administrative operations only.

MODULE 11

Aurora Advanced DBA Topics

Aurora provides several capabilities that have no equivalent in standard RDS PostgreSQL. These advanced features — Serverless v2, Parallel Query, and Global Database — require dedicated study because their operational behavior, cost models, and failure modes differ substantially from conventional database deployments. Misconfiguring any of these can result in either unexpected costs or degraded availability.

Aurora Serverless v2 Configuration

```
# CloudFormation Serverless v2 Scaling Configuration
ServerlessV2ScalingConfiguration:
  MinCapacity: 0.5   # Minimum (pause-capable)
  MaxCapacity: 128   # Maximum (128 ACUs)
```

ACU Capacity Planning by Workload

Workload Type	Min ACU	Max ACU	Notes
Development	0.5	2–4	Cost-optimized; pause when idle
Light Production	0.5–2	8–16	Must not be set to 0 if always-on required
Standard Production	2–4	32–64	Set min based on connection requirements
High Performance	8+	64–128	Pre-warm with higher minimum ACU

Aurora Parallel Query

Offloads analytical query processing to the distributed storage layer. Available for queries scanning more than 1GB of data. Configured via the `aurora_parallel_query` parameter. Reduces impact of analytical workloads on the writer instance buffer pool.

Aurora Global Database

Provides cross-region disaster recovery with less than 1 second RPO. Supports up to 5 secondary regions with managed failover including DNS updates. The replication is storage-level — no WAL shipping, no replica lag accumulation.

MODULE 12

Troubleshooting Playbook

Effective troubleshooting on managed PostgreSQL requires knowing which diagnostic surface to check first. Production incidents rarely give you time to think from scratch — having a pre-built mental model of where to look for each symptom class dramatically reduces mean time to resolution. The following playbook covers the most common production incident types.

Issue	Diagnostic	Resolution
High CPU	Performance Insights → Top SQL by CPU	Optimize queries, add indexes, scale up
Long-running queries	`pg_stat_activity` + `state_change`	Set `statement_timeout`, kill PID if needed
Lock contention	pg_locks + pg_stat_activity	Find blocker, evaluate `lock_timeout`
Replica lag	`pg_stat_replication`	Check network, WAL generation rate, storage I/O
Connection saturation	`DatabaseConnections` CloudWatch	Implement pooling, review `max_connections`
Disk full	`FreeStorageSpace` CloudWatch	Enable autoscaling, archive or purge old data
Autovacuum lag	`pg_stat_user_tables` dead tuples	Tune autovacuum, run manual VACUUM, consider partitioning

Aurora Serverless v2 Specific Issues

Cold Starts

Cold starts occur when the cluster is at minimum ACU (0.5) and receives a burst of connections. Mitigation: increase minimum ACU or maintain warm connections via a keep-alive query on a monitoring connection.

Scale Lag

Aurora Serverless v2 scales in increments and has a brief lag between ACU allocation decisions and actual capacity availability. Set realistic minimum ACU values and use RDS Proxy to absorb connection bursts during this window.

Unexpected Scaling

If the cluster is scaling higher than expected, analyze `pg_stat_statements` for resource-heavy queries that are consuming CPU or memory. Unexpected scaling is almost always caused by query inefficiency, not legitimate workload growth.

MODULE 13 ⭐ MOST IMPORTANT

Production Readiness Checklist

The production readiness checklist is the most operationally critical section of this entire curriculum. A system that passes all prior modules but fails this checklist is not production-ready. Work through each category systematically during pre-launch reviews and revisit quarterly as part of operational hygiene. Items marked with ⭐ represent absolute minimums — no production deployment should go live without them.

HA Architecture

- Multi-AZ or Aurora cluster configured
- RTO/RPO defined, documented, and tested
- RDS Proxy deployed for connection resilience
- Failover testing performed and scheduled monthly

DR Strategy

- Cross-region replicas or Aurora Global Database
- Snapshot copies to secondary region automated
- Documented runbook for DR failover scenarios

Backup Validation

- Automated backups enabled (maximum retention)
- Manual snapshot strategy for critical change points
- Quarterly restore testing to a separate environment
- PITR tested end-to-end and recovery time documented

Capacity Planning

- Baseline metrics established (CPU, memory, connections)
- Load testing completed at 2x expected peak
- Autoscaling policies configured and validated
- 12-month growth projection documented

Security

- Private subnet deployment — no public accessibility
- IAM authentication for all application connections
- SSL enforced via `rds.force_ssl=1`
- Encryption at rest with KMS enabled
- Audit logging via pgAudit or Database Activity Streams
- Secrets Manager rotation configured and tested

Monitoring

- CloudWatch alarms configured for all critical metrics
- Performance Insights enabled with 30-day retention
- Enhanced Monitoring enabled at ≤10s granularity
- Log retention set to ≥1 month
- `pg_stat_statements` configured and active

Key Metrics at a Glance

A consolidated view of the most critical performance and health metrics for production PostgreSQL on RDS and Aurora, including alert thresholds and the action each metric should trigger.

Buffer Cache Hit Ratio

Target >99% hit ratio. Below this threshold, queries are reading from disk rather than memory — investigate `shared_buffers` sizing or query bloat.

Max CPU Threshold

Alert at >80% sustained CPU utilization. Indicates compute saturation — escalate to query optimization or instance scale-up.

Min Freeable Memory

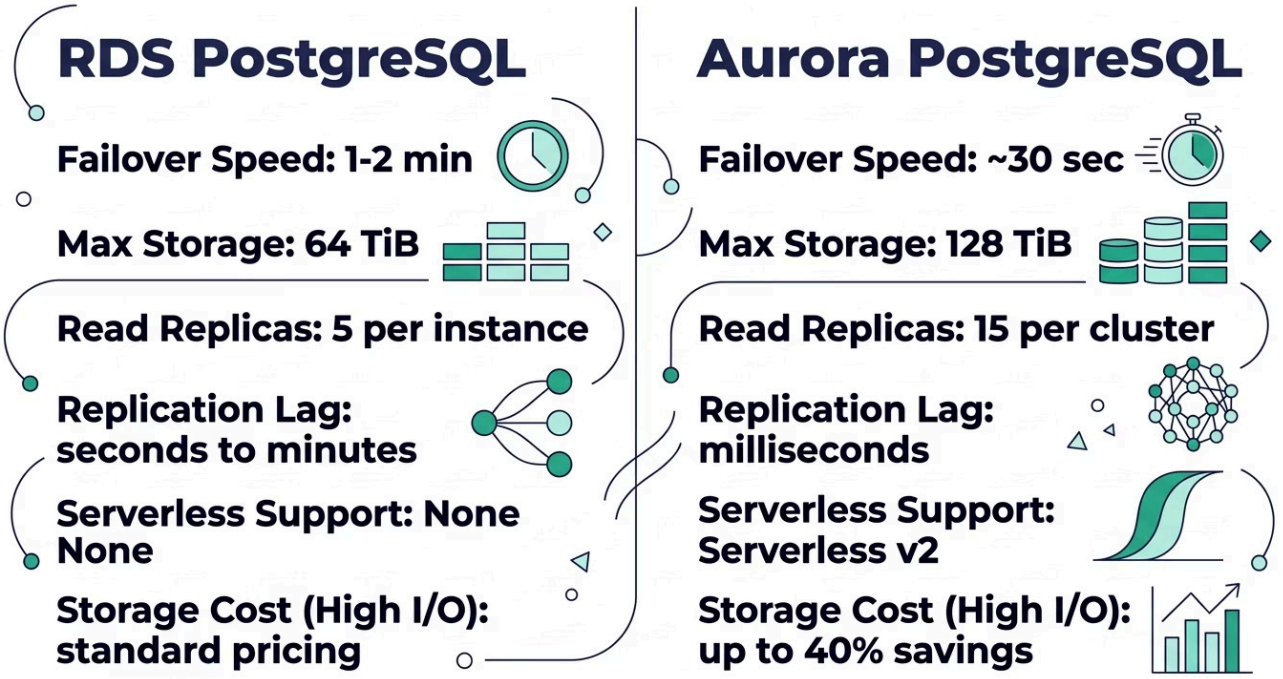
Alert when FreeableMemory drops below 10% of total instance RAM. Memory pressure increases OOM risk and degrades query plan quality.

Max Connection Usage

Alert at >80% of max_connections. Beyond this threshold, new connections may be rejected under burst load without pooling in place.

RDS vs. Aurora: Architecture Decision Guide

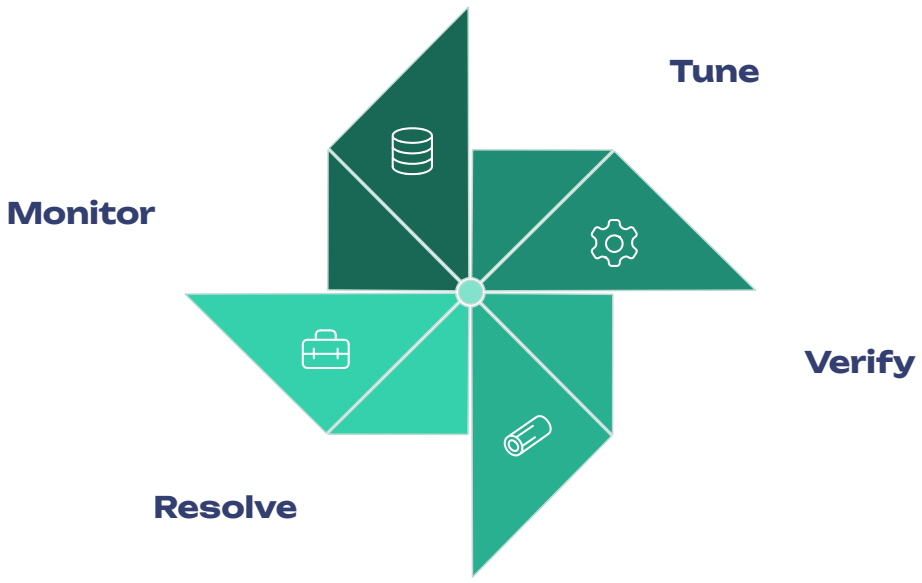
Choosing between RDS PostgreSQL and Aurora PostgreSQL is one of the most consequential architectural decisions for a managed database deployment. Use the following framework to evaluate the right service for your workload characteristics, budget constraints, and availability requirements.



Decision Rule of Thumb: Choose Aurora when you need sub-minute failover, more than 5 read replicas, cross-region DR with <1s RPO, or Serverless scaling. Choose RDS when workloads are simple, cost is the primary constraint, and you need direct control over EBS storage configuration.

Autovacuum Health at a Glance

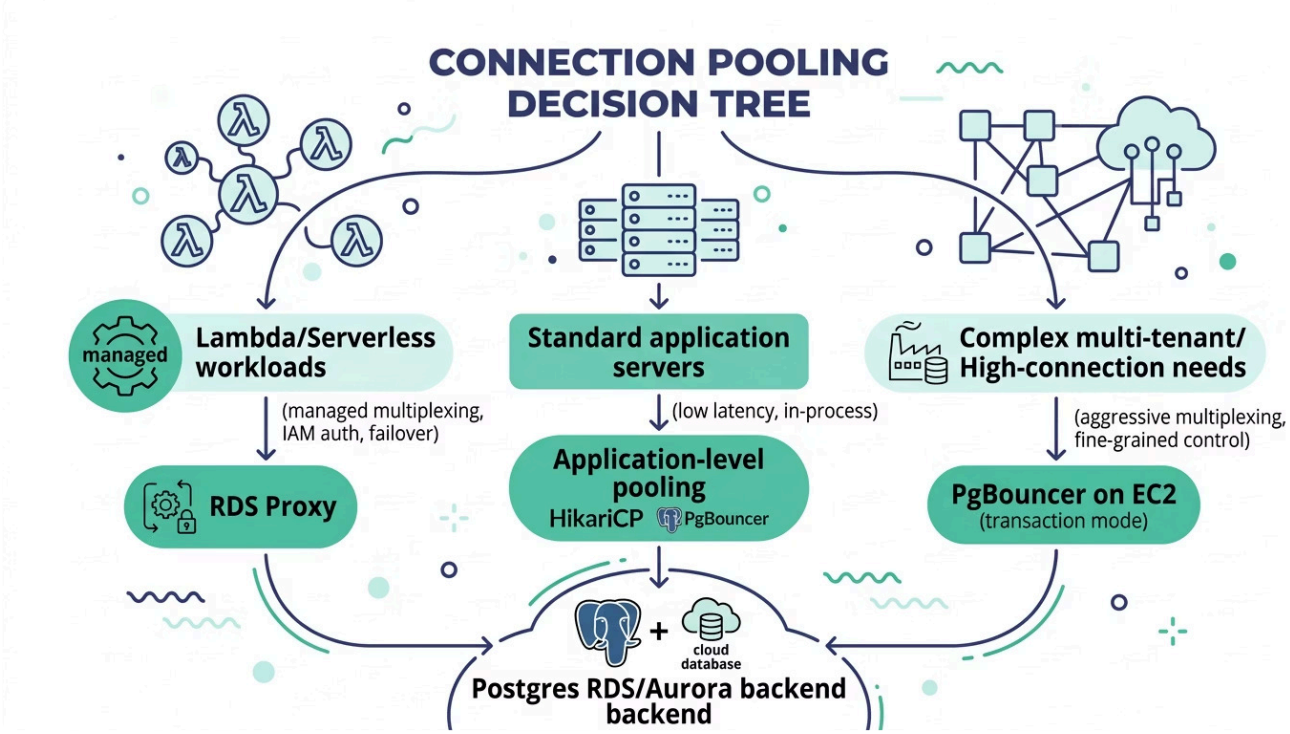
Autovacuum is the most frequently misunderstood PostgreSQL maintenance process. When properly configured, it runs invisibly in the background. When misconfigured, it manifests as table bloat, degraded query performance, and — in the worst case — transaction ID wraparound autovacuum, which forces a complete database freeze. Monitor and tune proactively.



The autovacuum health cycle should be part of every DBA's weekly operational review. Tables with dead_pct consistently above 10% are candidates for table-level parameter overrides. Tables approaching transaction ID age limits (200M XID threshold) require immediate manual intervention with `VACUUM FREEZE`. Never let a production table reach the forced-freeze threshold — the resulting autovacuum wrap-around will lock the table for minutes or hours depending on table size.

Connection Architecture Decision Tree

Selecting the right connection management architecture depends on your application type, traffic pattern, and deployment environment. Serverless and Lambda-based workloads have fundamentally different connection requirements than long-running application servers, and the wrong choice can result in connection storms, throttling, or unexpectedly high database process counts.



Regardless of the pooling architecture chosen, always monitor `DatabaseConnections` in CloudWatch against the instance's theoretical maximum. Set an alarm at 80% utilization and review connection pool sizing as part of every capacity planning cycle. Connection exhaustion events are almost always preceded by a period of gradual growth that CloudWatch trending would have made visible weeks in advance.

Security Defense-in-Depth Model

Production PostgreSQL security on AWS is not a single control but a layered system where each layer independently reduces risk. A failure at the network layer (e.g., a misconfigured security group) should not result in data exposure because authentication, encryption, and audit controls provide additional independent barriers. Build and validate each layer separately.

Backup Strategy Comparison

Understanding the backup architecture of each service is essential for accurately communicating RTO and RPO to stakeholders. The numbers below represent typical observed performance under standard conditions — actual recovery times vary with database size, network conditions, and the specific restore target.

RDS PostgreSQL Backup Model

Daily automated snapshot captures the full database once per day during the preferred backup window. Continuous WAL archiving fills the gaps, enabling PITR to any second within the retention period.

Recovery process: Restore the most recent snapshot, then replay WAL to the target recovery point. Recovery time scales with the amount of WAL to replay — a large, write-heavy database recovered 12 hours forward from a daily snapshot may take 30–90+ minutes.

Performance impact: Slight I/O impact on single-AZ instances during the backup window. Multi-AZ instances perform backups from the standby with zero primary impact.

Aurora PostgreSQL Backup Model

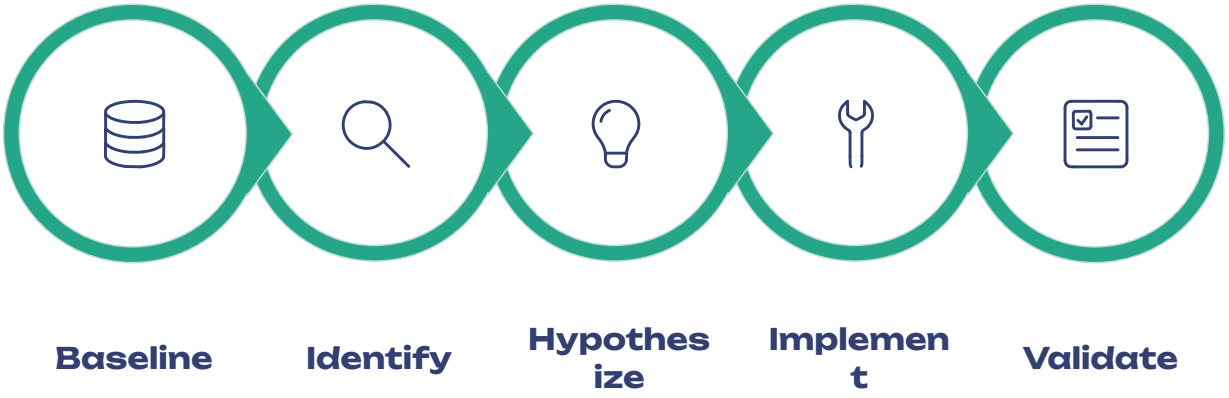
Continuous storage-level incremental snapshots are taken automatically without any performance impact on the writer instance. There is no dedicated backup window — protection is always active.

Recovery process: Aurora PITR is significantly faster than RDS because the storage layer directly reconstructs the requested point-in-time state without replaying WAL. Large databases can often be restored in minutes rather than hours.

Aurora Backtrack: For operational mistakes (accidental DELETE or DROP), Backtrack allows rewinding the cluster to a specific point in time within the last 72 hours without a full restore — typically completing in seconds to minutes.

Performance Tuning Workflow

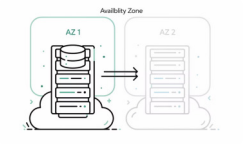
A disciplined performance tuning workflow prevents the most common DBA mistake: making configuration changes based on intuition rather than evidence. Always measure before and after any change, and change only one variable at a time. The following workflow has been validated across hundreds of production PostgreSQL tuning engagements.



The most common failure mode in PostgreSQL performance tuning is skipping the baseline step and jumping directly to implementation. Without a quantitative before-state, it is impossible to measure whether a change actually improved performance or merely shifted the bottleneck. Use `pg_stat_statements` snapshots (reset with `pg_stat_statements_reset()` before and after a change window) to generate reproducible before/after comparisons. Document every change and its measured outcome — this builds institutional knowledge that accelerates future tuning cycles.

High Availability Architecture Patterns

The right HA architecture depends on your RTO requirements, budget, and workload read/write ratio. The three primary patterns below represent increasing levels of complexity, cost, and availability guarantees. Most production workloads should target Pattern 2 or Pattern 3.



Pattern 1: RDS Multi-AZ Single Standby

Synchronous replication to one standby in a separate AZ. Automatic failover in 1–2 minutes. No read traffic on standby. Lowest cost HA option — suitable for non-critical applications with moderate RTO tolerance.

Pattern 2: Aurora Cluster (Writer + Readers)

Six-way storage replication across 3 AZs. Up to 15 reader endpoints sharing the same storage volume. ~30 second failover with automatic promotion of a reader to writer. Best for read-heavy production workloads needing sub-minute recovery.

Pattern 3: Aurora Global Database

Primary cluster in one region with up to 5 secondary read-only clusters in other regions. Less than 1 second RPO for cross-region DR. Storage-level replication with no performance impact on the primary. Required for regulated industries or global applications with strict DR SLAs.

Aurora Serverless v2: Operational Patterns

Aurora Serverless v2 represents a paradigm shift in database capacity management — moving from fixed instance sizing to continuous, fine-grained ACU allocation. While this eliminates the over-provisioning problem, it introduces new operational considerations around cold starts, scale-up lag, and connection management that require deliberate configuration to avoid production incidents.

1 Define ACU Range

Set MinCapacity based on connection requirements and warm-cache needs, not cost alone. Set MaxCapacity as a cost ceiling with clear alerting at 80% of max.

2 Establish Baseline

Run representative load tests and observe the ACU consumption profile. This establishes the minimum ACU needed to serve baseline traffic without scale-up lag during normal operations.

3 Deploy RDS Proxy

Place RDS Proxy in front of all Serverless v2 clusters. This buffers connection bursts during scale-up transitions and provides connection multiplexing benefits.

4 Monitor & Tune

Use Performance Insights to identify queries triggering unexpected scale-up events. Optimize those queries to reduce ACU consumption and control costs.

ACU Cost Model: Aurora Serverless v2 charges per ACU-hour. At low traffic periods, the cluster scales down to MinCapacity. Setting MinCapacity to 0.5 ACUs enables near-pause behavior for dev/test environments, while production clusters should typically maintain a MinCapacity of 2–4 ACUs to avoid cold-start latency affecting user-facing traffic.

READY FOR PRODUCTION

Summary: PostgreSQL RDS/Aurora Mastery

This learning path covers the complete spectrum of skills required for production PostgreSQL DBA work on AWS — from foundational instance management through expert-level troubleshooting and architecture design. The modules are sequenced to build knowledge progressively, but each section also stands alone as a reference for day-to-day operational work.

🏗️ Instance & Config

Modules 1–2: RDS vs. Aurora architecture, storage types, memory parameters, and Aurora-specific tuning considerations.

🔄 HA & Replication

Modules 3–4: Multi-AZ options, Aurora 6-way replication, RDS Proxy failover optimization, and read scaling patterns.

💾 Backup & Performance

Modules 5–6: Backup architecture, PITR strategies, query tuning workflow, index optimization, and pg_stat_statements analysis.

📊 Monitoring & Ops

Modules 7–8: Performance Insights, CloudWatch alerting thresholds, Enhanced Monitoring, autovacuum tuning, and bloat remediation.

🔐 Security & Scale

Modules 9–10: Connection pooling strategies, RDS Proxy, IAM authentication, encryption, audit logging, and least-privilege access control.

🚀 Advanced & Production

Modules 11–13: Aurora Serverless v2, Global Database, Parallel Query, troubleshooting playbook, and the complete production readiness checklist.

> The Troubleshooting Playbook and Production Readiness Checklist in Modules 12–13 are particularly critical for real-world DBA work. Master those two sections first if you are preparing for an imminent production deployment.